

Indexing

Clustering vs. Unclustering

General

Feature	Clustered Index	Unclustered Index
Data Order	Physically orders the rows in the table by the indexed column(s).	Leaves the table rows unordered; index stores pointers to actual rows.
Storage	There is only one clustered index per table.	You can have multiple unclustered indexes.
Index Structure	The index is the table — data is stored in the B+-tree itself.	The index is a separate structure pointing to data rows.
Access Speed	Fast for range scans or sorted results (e.g., <code>BETWEEN</code> , <code>ORDER BY</code>).	Fast for point queries if highly selective. Slower for range scans (extra lookups).
Insert/Update Cost	Higher, because data may need to be moved to maintain order.	Lower, as row order is not affected.
Example	<code>CLUSTERED INDEX ON date</code> → rows in table are physically sorted by date.	<code>UNCLUSTERED INDEX ON userID</code> → index maps userIDs to row pointers in heap.

When to use which?

Use Case	Best Index Type
Point lookup (<code>id = 123</code>)	Unclustered is fine if selective
Range queries (<code>x > 100</code>)	Clustered is faster
Sorting (<code>ORDER BY x</code>)	Clustered avoids extra sort
Covering queries	Either, depending on columns covered
<code>SELECT *</code> (many rows)	Clustered may reduce I/O due to spatial locality

Index Selection Cheat Sheet

General Strategy

Ask yourself:

1. **Is there a `WHERE` clause?** → Look for indexes on the filtering column(s).
2. **Is there `GROUP BY` or `ORDER BY` ?** → Look for indexes on the grouped/sorted column(s).
3. **Is there aggregation (`AVG` , `COUNT` , etc.) or `SELECT *` ?** → Think about access paths and whether index-only scans are possible.
4. **Is the index clustered or unclustered?** → Unclustered = good for selective filters, bad for full table scans.

When to Use an Index

Query Pattern	Best Index
<code>WHERE col = value</code>	Index on <code>col</code> (high selectivity = better)
<code>WHERE col1 = x AND col2 = y</code>	Index on <code>(col1, col2)</code> if available; otherwise, index on most selective column
<code>GROUP BY col</code>	Index on <code>col</code>
<code>ORDER BY col</code>	Index on <code>col</code>
<code>SELECT *</code> with no filtering	No index (full table scan better than using unclustered index)
<code>SELECT AVG(col)</code> with <code>WHERE</code>	Index on filtering column(s); <code>col</code> can be unindexed
<code>WHERE col BETWEEN x AND y</code>	Index on <code>col</code> (range scan)

Index Tips

- **Unclustered B+-trees:** Good for **point lookups** or **small result sets**. Bad for scanning large portions.
- **Avoid indexes** when:
 - The query returns **most** rows.
 - There's no filtering or grouping.
- **Composite Index (col1, col2):**

- Usable **only if** `col1` is part of the condition.
- `WHERE col2 = x` → **Cannot use** index `(col1, col2)` directly.

Selectivity Rule of Thumb

- High cardinality (many distinct values) → Index is **very useful**
 - E.g., `messageID`, `senderID`, `receiverID` (10 million values)
- Low cardinality (few distinct values) → Index is **less useful**
 - E.g., if `status` has only 3 values

Common Mistakes

- Using index on `messageID` when it's not in the query
- Thinking index on `col2` helps in `(col1, col2)` composite index without using `col1`
- Choosing an index for a full scan query

Quick Decision Guide

Situation	Action
<code>WHERE date = x</code>	Use <code>Messages(date)</code>
<code>GROUP BY date</code>	Use <code>Messages(date)</code>
<code>SELECT *</code>	Use no index
<code>WHERE senderID = x AND receiverID = y</code>	Use <code>Messages(senderID)</code> or <code>Messages(receiverID)</code> (whichever is more selective)
<code>SELECT AVG(size)</code> with no filter	Use no index
<code>GROUP BY col</code> + aggregation	Use <code>Messages(col)</code>